

# Linux Device Management - Study Notes

## I. /dev Directory

### Overview

- **Purpose:** Contains device files (device nodes) that allow interaction with device drivers
- **Nature:** Special files that look like regular files but represent hardware devices
- **Usage:** Can interact with these files using standard commands like `ls`, `cat`, etc.

### Key Concepts

- Device drivers are accessed through device files stored in `/dev`
- `/dev/null` is a common example - discards all input and returns nothing

### Historical Problems

- **Old method:** Manually add device files to `/dev` directory
- **Issues:**
  - Directory became cluttered with unused device files
  - Devices assigned files based on discovery order (inconsistent across reboots)
  - Static device files remained even after devices were removed

### dd Command Example

```
$ ls /dev
# Shows all device files on the system
```

## 2. Device Types

### File Permissions Format

```
$ ls -l /dev

brw-rw---- 1 root disk      8,  0 Dec 20 20:13 sda
crw-rw-rw- 1 root root      1,  3 Dec 20 20:13 null
```

```
srw-rw-rw- 1 root root      0 Dec 20 20:13 log
prw-r--r-- 1 root root      0 Dec 20 20:13 fdata
```

## Column Structure (left to right)

1. Permissions
2. Owner
3. Group
4. Major Device Number
5. Minor Device Number
6. Timestamp
7. Device Name

## Device Type Indicators (first character)

- **c** - Character device
- **b** - Block device
- **p** - Pipe device
- **s** - Socket device

## Device Categories

### Character Devices

- **Transfer**: Data one character at a time
- **Examples**: `/dev/null`, pseudo devices
- **Nature**: Often not physically connected but provide OS functionality

### Block Devices

- **Transfer**: Data in large fixed-sized blocks
- **Examples**: Hard drives, filesystems
- **Use case**: Devices that work with data blocks

### Pipe Devices

- **Purpose**: Allow communication between two or more processes
- **Function**: Similar to character devices but output goes to another process

### Socket Devices

- **Purpose**: Facilitate communication between processes

- **Advantage:** Can communicate with many processes simultaneously

## Device Characterization

- **Major Device Number:** Represents the device driver (e.g., 8 for sd block devices)
- **Minor Device Number:** Identifies unique device within driver class (e.g., 0 for first device)

**Example:** Device numbers `8, 0`

- Major: 8 (sd block device driver)
- Minor: 0 (first device, represented as 'a')

## 3. Device Names

### SCSI Device Naming

- **Protocol:** SCSI (Small Computer System Interface) - pronounced "scuzzy"
- **Usage:** Mass storage devices, printers, scanners, peripherals
- **Prefix:** `sd` (SCSI disk)

### Common SCSI Device Files

```
/dev/sda    # First hard disk
/dev/sdb    # Second hard disk
/dev/sda3   # Third partition on first hard disk
```

### Pseudo Devices

- **Nature:** Not physically connected to system
- **Type:** Usually character devices

### Common Pseudo Devices

```
/dev/zero    # Accepts/discards input, produces continuous NULL bytes
/dev/null    # Accepts/discards input, produces no output
/dev/random  # Produces random numbers
```

### PATA Devices (Legacy)

- **Prefix:** `hd` (found in older systems)
- **Examples:**

```
/dev/hda      # First hard disk
/dev/hdd2     # Second partition on 4th hard disk
```

## 4. sysfs

### sysfs Overview

- **Purpose:** Better device management than `/dev` directory
- **Type:** Virtual filesystem
- **Mount point:** Usually `/sys`

### Key Differences from `/dev`

- `/dev`: Simple access to devices themselves
- `/sys`: View information and manage devices (not device nodes)

### Information Provided

- Manufacturer and model details
- Device connection location
- Device state
- Device hierarchy
- And much more

### Example

```
pete@icebox:~$ ls /sys/block/sda
```

|                  |                   |          |           |           |        |
|------------------|-------------------|----------|-----------|-----------|--------|
| alignment_offset | discard_alignment | holders  | removable | sda6      | trace  |
| bdi              | events            | inflight | ro        | size      | uevent |
| capability       | events_async      | power    | sda1      | slaves    |        |
| dev              | events_poll_msecs | queue    | sda2      | stat      |        |
| device           | ext_range         | range    | sda5      | subsystem |        |

## 5. udev

### Traditional Method (Manual)

### Creating Device Nodes

```
$ mknod /dev/sdb1 b 8 3
# Creates device node /dev/sdb1 as block device (b) with major 8, minor 3
```

## Removing Device Nodes

```
rm /dev/sdb1 # Simply remove the device file
```

## Modern udev System

- **Function:** Dynamically creates and removes device files
- **Daemon:** `udev` runs continuously
- **Process:** Listens for kernel messages about connected devices

## udev Workflow

1. `udev` receives kernel messages about devices
2. Parses device information
3. Matches data with rules in `/etc/udev/rules.d`
4. Creates device nodes and symbolic links based on rules

## udev Management

### View udev Database

```
$ udevadm info --query=all --name=/dev/sda
# Shows detailed information about /dev/sda
```

## 6. Device Listing Commands

### USB Devices

```
$ lsusb
# Lists all USB devices
```

### PCI Devices

```
$ lspci
# Lists all PCI devices
```

# SCSI Devices

```
$ ls SCSI
# Lists all SCSI devices
```

## 7. dd Command

### dd Command Overview

- **Purpose:** Convert and copy data
- **Function:** Reads from input file/stream, writes to output file/stream

### Basic Syntax

```
$ dd if=/home/pete/backup.img of=/dev/sdb bs=1024
# Copies backup.img to /dev/sdb in 1024-byte blocks
```

### Key Parameters

- **if=file:** Input file (source)
- **of=file:** Output file (destination)
- **bs=bytes:** Block size (can use k, m, etc. for kilobytes, megabytes)
- **count=number:** Number of blocks to copy

### Advanced Example

```
$ dd if=/home/pete/backup.img of=/dev/sdb bs=1M count=2
# Copies only 2 megabytes (2 blocks of 1M each) from a 10M file
```

### Important Notes

- **Power:** Can backup entire disk drives, restore disk images
- **Caution:** Extremely powerful tool - can cause data loss if misused
- **Optimization:** `bs` and `count` parameters can optimize data transfers
- **Default behavior:** Without `count`, copies entire file

### Use Cases

- Creating backups of disks/partitions
- Restoring disk images

- Copying data between devices
- Creating disk images